# Day 14 – LLM Fundamentals: Tokenization, Training, Inference and Model Families

## 1. The complete LLM mental model

An LLM application follows this flow:

```
Raw text
   ↓
Tokenizer
   ↓
Token IDs
   ↓
Transformer predicts probabilities for the next token
   ↓
Decoding parameters select a token
   ↓
Selected token is added to the sequence
   ↓
Process repeats until completion
   ↓
Tokens are converted back into text
```

An LLM does not directly process sentences or words. It processes **token IDs** and repeatedly predicts:

> "Given all previous tokens, what token is most likely to come next?"

That simple objective, when trained at enormous scale, produces capabilities such as summarization, question answering, coding and reasoning.

---

# 2. Tokenization

## 2.1 What is a token?

A **token** is a unit of text understood by the model.

A token may be:

- A complete word
- Part of a word
- A punctuation mark
- A space combined with a word
- A number or part of a number
- Sometimes an individual byte or character

For example, a tokenizer might split:

```
"Transformers are unbelievable!"
```

into something conceptually similar to:

```
["Transform", "ers", " are", " un", "believ", "able", "!"]
```

The actual split depends on the tokenizer.

Tokens are therefore **not the same as words or characters**. OpenAI's tokenizer guidance notes that a token can range from a character fragment to a complete word; as a rough English-only approximation, one token is often around four characters or three-quarters of a word. This approximation is unreliable for code, numbers and many non-English languages. (OpenAI Help Center)

## 2.2 Why not use complete words?

A word-level vocabulary creates problems:

```
run
running
runner
rerunning
```

Treating these as completely independent words creates a very large vocabulary and makes rare or unseen words difficult to handle.

Character-level tokenization solves the unknown-word problem but produces very long sequences:

```
"running" → ["r", "u", "n", "n", "i", "n", "g"]
```

Subword tokenization provides a compromise:

```
"running" → ["run", "ning"]
"rerunning" → ["re", "run", "ning"]
```

The model can represent rare words using known pieces while keeping common words compact.

---

## 2.3 Byte Pair Encoding, or BPE

BPE is a common subword-tokenization idea.

### Intuition

1. Begin with small units, such as characters or bytes.
2. Count which adjacent units occur together most frequently.
3. Merge the most frequent pair.
4. Repeat until the desired vocabulary size is reached.

Suppose the training data repeatedly contains:

```
l o w
l o w e r
l o w e s t
```

The tokenizer may learn merges such as:

```
l + o → lo
lo + w → low
```

Eventually, common sequences become individual tokens:

```
"low"    → ["low"]
"lowest" → ["low", "est"]
```

BPE tends to retain frequent words or fragments while splitting rare words into smaller pieces. It became a widely used method for handling rare and out-of-vocabulary words in neural language systems. (ACL Anthology)

### Important interview point

BPE is learned from the tokenizer's training corpus. Therefore, two models can tokenize the same sentence differently.

---

## 2.4 SentencePiece

SentencePiece is better understood as a **tokenization framework**, not one specific segmentation algorithm.

It can train tokenizers directly from raw text and commonly supports algorithms such as:

- BPE
- Unigram language-model tokenization

Traditional tokenizers often split text into words before building subwords. SentencePiece can learn directly from raw sentences, including whitespace information, making it useful for multilingual and language-independent processing. (arXiv)

### BPE versus SentencePiece

| Concept | BPE | SentencePiece |
|---|---|---|
| What is it? | A subword-merging algorithm | A tokenizer framework/library |
| Main idea | Repeatedly merge frequent symbol pairs | Learn subwords directly from raw text |
| Algorithms | Primarily BPE | BPE, Unigram and related methods |
| Pre-tokenization | Often uses some preprocessing | Can work directly on raw sentences |
| Multilingual use | Possible | Designed to be language-independent |

A frequent interview mistake is saying:

> "BPE and SentencePiece are two completely different tokenization algorithms."

A better answer is:

> "BPE is a tokenization algorithm, while SentencePiece is a framework that can implement BPE or Unigram-based tokenization."

---

## 2.5 Why tokenization matters in production

Tokenization directly affects four things:

### Context usage

A poorly tokenized input consumes more context-window space.

```
Same information
→ more tokens
→ less room for conversation, documents and output
```

### Cost

Most hosted APIs bill according to input and output tokens.

```
More tokens → higher API cost
```

### Latency

More input tokens require more prompt processing. More output tokens require more autoregressive generation steps.

### Multilingual performance

Some languages may require significantly more tokens than English for equivalent information. This can increase cost and reduce the effective amount of information that fits inside the context window.

### RAG chunking

RAG chunks should normally be sized in **tokens**, not only characters or words.

A chunk of 1,000 English words and a chunk of 1,000 words in another language may produce very different token counts.

---

# 3. LLM training pipeline

A useful high-level pipeline is:

```
Pre-training
     ↓
Supervised fine-tuning / instruction tuning
     ↓
Preference alignment: RLHF or DPO
     ↓
Safety testing and evaluation
     ↓
Deployment and inference
```

These stages are related but solve different problems.

---

## 3.1 Pre-training

During pre-training, the model learns from a massive corpus that may contain:

- Web documents
- Books
- Articles
- Code
- Scientific material
- Curated datasets

For a decoder-only LLM, the typical objective is **next-token prediction**.

Input:

```
The capital of France is
```

Target:

```
Paris
```

More precisely, the model predicts the next token at every position:

```
The       → capital
The capital → of
The capital of → France
...
```

The model's predicted probability distribution is compared with the correct next token using a loss function, commonly cross-entropy loss. Backpropagation and an optimizer update billions of parameters.

### What pre-training teaches

Pre-training develops:

- Language patterns
- Grammar
- Broad factual associations
- Code patterns
- Semantic relationships
- Some reasoning patterns
- General completion ability

### What it does not guarantee

A pre-trained model may not reliably:

- Follow user instructions
- Refuse unsafe requests
- Produce concise answers
- Maintain a helpful conversational format

It has primarily learned to **continue text**, not necessarily to behave like an assistant.

---

## 3.2 Fine-tuning

Fine-tuning continues training an existing model on a smaller, more targeted dataset.

Examples include:

- Legal-document language
- Medical terminology
- Customer-support conversations
- SQL generation
- Code completion
- Company-specific writing patterns

Fine-tuning changes model behaviour or domain capability by updating some or all model parameters.

### Full fine-tuning versus parameter-efficient fine-tuning

**Full fine-tuning** updates most or all model weights.

**Parameter-efficient fine-tuning**, such as LoRA, updates a much smaller set of additional parameters.

The latter generally requires less compute and storage, although it may not provide the same flexibility as full training for every task.

---

## 3.3 Instruction tuning

Instruction tuning is a particular kind of supervised fine-tuning.

The model is trained on instruction-and-response examples:

```
Instruction:
Summarize the following incident report.

Response:
The service failed because...
```

Other examples might include:

```
Instruction: Translate this into Hindi.
Response: ...

Instruction: Extract the invoice number.
Response: ...

Instruction: Write a Python function.
Response: ...
```

Instruction tuning teaches the model:

> "When a user gives an instruction, produce an appropriate response."

### Fine-tuning versus instruction tuning

- **Fine-tuning** is the broad category.
- **Instruction tuning** is fine-tuning specifically on instruction-response data.

---

## 3.4 RLHF

RLHF means **Reinforcement Learning from Human Feedback**.

A simplified RLHF pipeline is:

```
1. Generate multiple candidate answers
2. Humans rank or compare the answers
3. Train a reward model from those preferences
```

```
4. Use reinforcement learning to optimize the LLM
5. Keep the updated model reasonably close to the original model
```

For example:

```
Prompt: Explain gravity to a child.

Response A: Clear, friendly explanation
Response B: Complicated mathematical explanation

Human preference: A > B
```

The reward model learns to assign a higher score to responses resembling A.

RLHF is intended to improve qualities such as:

- Helpfulness
- Instruction following
- Safety
- Conversational behaviour
- Alignment with human preferences

Research behind InstructGPT showed that supervised instruction tuning followed by human-feedback-based optimization can make language models better aligned with user intent. (arXiv)

### RLHF challenges

RLHF can be operationally complex because it may require:

- High-quality human rankings
- A separately trained reward model
- Generation of many candidate responses
- Reinforcement-learning infrastructure
- Careful control against reward hacking and model drift

---

## 3.5 DPO

DPO means **Direct Preference Optimization**.

It also uses preference data:

```
Prompt
Preferred response
Rejected response
```

However, instead of explicitly training a reward model and then running an RL algorithm, DPO directly increases the probability of the preferred response relative to the rejected response.

```
Preferred answer   → make more likely
Rejected answer    → make less likely
```

The original DPO work presents it as a simpler alternative that avoids an explicit reward-model-and-RL training loop. (arXiv)

### RLHF versus DPO

| Aspect | Traditional RLHF | DPO |
|---|---|---|
| Input data | Human preferences | Human or model-generated preferences |
| Reward model | Usually explicit | Not explicitly required |
| Reinforcement learning | Yes | No separate RL loop |
| Complexity | Higher | Generally simpler |
| Main purpose | Preference alignment | Preference alignment |
| Risks | Reward hacking, instability | Dataset bias, overfitting to preferences |

### Interview-ready training answer

> “Pre-training teaches broad language and world patterns through next-token prediction. Supervised instruction tuning teaches the model how to follow requests. Preference optimization, through RLHF or DPO, then makes preferred responses more likely and improves helpfulness, safety and style.”

---

# 4. LLM inference

## 4.1 What happens during inference?

Suppose the prompt is:

```
The best way to reduce API latency is
```

The model produces probabilities for possible next tokens:

```
"to"       0.40
"by"       0.25
"through"  0.15
"caching"  0.10
...
```

A decoding strategy chooses one token.

The chosen token is added to the input, and the process repeats:

```
Prompt → token 1 → token 2 → token 3 → ...
```

Because decoder-only LLMs generate tokens sequentially, long responses generally take longer to produce.

---

# 5. Inference parameters

## 5.1 Temperature

Temperature modifies the shape of the next-token probability distribution.

### Lower temperature

```
Temperature: approximately 0–0.2
```

- More predictable
- More focused
- Less varied
- Better for extraction, classification and structured output

### Higher temperature

```
Temperature: approximately 0.7–1.0+
```

- More varied
- More creative
- Greater chance of surprising output
- Potentially greater chance of irrelevant or incorrect output

Conceptually:

```
Original probabilities:
A = 0.60
B = 0.25
C = 0.15

Low temperature:
A becomes much more dominant

High temperature:
The probabilities become more balanced
```

Temperature changes the sampling distribution; it does not increase the model's knowledge.

Also, temperature zero should not be treated as a guarantee of perfect reproducibility. Infrastructure, model-version and numerical differences can still affect outputs.

---

## 5.2 Top-k

Top-k sampling keeps only the k most probable candidate tokens.

Suppose:

```
A = 0.40
B = 0.25
C = 0.15
D = 0.10
E = 0.10
```

With:

```
top-k = 3
```

Only A, B and C remain eligible.

### Effects

- Small k: focused and conservative
- Large k: more possible candidates and variation
- k = 1: equivalent to choosing the most likely token at each step

---

## 5.3 Top-p

Top-p is also called **nucleus sampling**.

Instead of keeping a fixed number of tokens, it keeps the smallest set whose cumulative probability reaches the chosen threshold.

Example:

```
A = 0.45   cumulative = 0.45
B = 0.25   cumulative = 0.70
C = 0.15   cumulative = 0.85
D = 0.10   cumulative = 0.95
E = 0.05   cumulative = 1.00
```

With:

```
top-p = 0.85
```

The model samples only from A, B and C.

### Top-k versus top-p

- Top-k uses a fixed number of candidate tokens.
- Top-p dynamically changes the number according to the probability distribution.

Official Transformers documentation defines top-k as retaining the highest-probability k tokens and top-p as retaining the smallest probability mass reaching the specified threshold. (Hugging Face)

---

## 5.4 Repetition penalty

A repetition penalty reduces the likelihood of tokens that have already appeared.

It can help prevent output such as:

```
The service failed because the service failed because the service failed...
```

But an excessive penalty can cause the model to:

- Avoid necessary technical terminology
- Use awkward synonyms
- Produce unnatural text
- Fail to repeat identifiers or names that must remain exact

Use it carefully for code, JSON, legal language and structured outputs, where repetition may be valid.

---

## 5.5 Maximum tokens

`max_tokens`, or in some libraries `max_new_tokens`, limits the number of tokens generated.

It is a **cap**, not a requested response length.

```
max_new_tokens = 500
```

means:

> "Generate no more than 500 additional tokens."

The model may stop sooner when it:

- Produces an end-of-sequence token
- Encounters a configured stop sequence
- Completes the requested answer
- Reaches another platform-specific limit

### Product effects

A smaller output limit provides:

- Lower maximum cost
- Lower worst-case latency
- Less risk of runaway responses

But setting it too low may cut off:

- JSON
- Code
- Explanations
- Tool arguments
- Final conclusions

---

## 5.6 Practical parameter presets

These are starting points, not universal rules.

| Product task | Temperature | Top-p | Output limit |
|---|---|---|---|
| Entity extraction | 0–0.2 | Usually broad/default | Strict |
| RAG factual answer | 0–0.3 | Usually broad/default | Moderate |
| Code generation | 0–0.3 | Model-dependent | Sufficient for code |
| Customer-support reply | 0.2–0.6 | Moderate | Controlled |
| Brainstorming | 0.7–1.0 | Moderate/high | Flexible |
| Creative writing | 0.8–1.2 | Moderate/high | Flexible |

### Important interview advice

Avoid adjusting temperature, top-k and top-p randomly at the same time.

Start with:

1. A clear task objective
2. A strong prompt
3. One decoding strategy

4. An evaluation dataset
5. Controlled parameter experiments

---

# 6. Creativity versus determinism

A useful mental model is:

```
Low temperature + restrictive sampling
→ focused and repeatable

High temperature + broad sampling
→ diverse and creative
```

But inference parameters do not fix:

- Missing knowledge
- Poor retrieval
- Ambiguous instructions
- Weak model capability
- Incorrect source documents
- Bad tool results

For a factual enterprise RAG system, quality usually depends more on:

```
Retrieval quality
+ context quality
+ prompt design
+ model capability
+ validation
```

than on finding a “perfect” temperature.

---

# 7. Context window

## 7.1 Definition

The context window is the amount of tokenized information the model can consider during a request.

It may contain:

```
System instructions
+ conversation history
+ user prompt
+ retrieved documents
+ tool outputs
+ generated response
```

Provider APIs may describe input limits and output limits separately, so always verify the exact model specification. Current model catalogs commonly publish both context-window and maximum-output-token values. (OpenAI Developers)

## 7.2 Example

Suppose a model permits:

```
Total usable context: 32,000 tokens
Input currently used: 28,000 tokens
Reserved output: 4,000 tokens
```

There is no remaining capacity.

Trying to request an additional 8,000-token answer may produce:

- A validation error
- Input truncation
- A shorter allowed output
- Provider-specific automatic context management

---

## 7.3 Truncation

Truncation removes tokens when the prompt exceeds the accepted limit.

Common strategies include:

### Remove the oldest messages

Useful for casual chat, but it may remove an important earlier requirement.

### Remove the newest or excess document text

This may cause an incomplete answer because relevant evidence disappears.

### Summarize previous conversation

Reduces token count but may lose fine-grained detail.

### Sliding window

Keep only the most recent portion of the conversation.

### Retrieval-based memory

Store older information externally and retrieve only relevant memories.

Some APIs automatically drop older conversation content after the input limit is exceeded, but the exact behaviour is provider- and endpoint-specific. (OpenAI Platform)

---

## 7.4 Long-context limitations

A large context window does **not** mean the model will use every token equally well.

Problems can include:

- Important information being buried in the middle
- Contradictory documents
- Irrelevant details distracting the model
- Increased prompt-processing latency
- Higher token cost
- Greater memory requirements

- Reduced output space
- Harder debugging and citation verification

This is sometimes described as the **lost-in-the-middle** problem: models may pay more attention to information near the beginning or end than to information buried inside a very long prompt.

### Does long context replace RAG?

Usually, no.

Long context and RAG solve different problems:

| Long context | RAG |
|---|---|
| Allows more content in one request | Selects relevant content |
| Useful for analyzing a known document set | Useful for searching a large changing corpus |
| Can include irrelevant information | Attempts to filter irrelevant information |
| Cost rises with included tokens | Cost depends on retrieved subset |
| Does not automatically ensure freshness | Can retrieve current indexed data |

A good production system may use both:

```
RAG selects relevant evidence
+
Long context accommodates the selected evidence
```

---

# 8. Popular model families

Version numbers change quickly. In interviews, focus on the family's deployment model, size, openness, strengths and operational trade-offs rather than memorizing every current version.

## 8.1 GPT family

Developed by OpenAI.

Typical characteristics:

- Hosted proprietary models
- Strong general-purpose instruction following
- Reasoning, coding and tool-use options
- Multimodal capabilities in applicable models
- Different tiers for quality, latency and cost
- Managed API deployment

OpenAI's current model catalog presents multiple GPT tiers with different intelligence, latency, context and cost characteristics. (OpenAI Developers)

**Good fit:** teams that want strong managed capabilities without operating model infrastructure.

**Trade-off:** less control over weights and serving internals, along with provider dependency.

---

## 8.2 Llama family

Developed by Meta.

Typical characteristics:

- Open-weight models
- Strong self-hosting ecosystem
- Can be fine-tuned and quantized
- Available through multiple clouds and inference platforms
- Useful when data control or deployment flexibility matters

Meta's Llama family includes pretrained and instruction-tuned models, with recent generations expanding into multimodal and long-context use cases. (AI Meta)

**Good fit:** private deployment, custom fine-tuning, research and infrastructure-controlled environments.

**Trade-off:** the organization becomes responsible for GPUs, scaling, security, upgrades and monitoring.

---

## 8.3 Mistral family

Developed by Mistral AI.

Typical characteristics:

- Open-weight and commercial offerings
- Strong emphasis on efficient models
- Dense and mixture-of-experts architectures
- Hosted and self-deployed options
- General, coding and multimodal variants

Mistral officially provides both open-weight and commercial model options across different performance and deployment profiles. (Mistral AI)

**Good fit:** organizations wanting a combination of deployment control and commercial support.

---

## 8.4 Gemma family

Developed by Google DeepMind.

Typical characteristics:

- Lightweight open-weight models
- Multiple parameter sizes
- Suitable for local, edge and cloud deployment
- General-purpose and specialized variants
- Built using research and technology related to Google's Gemini ecosystem

Google describes Gemma as a collection of lightweight open models intended for customizable deployment across different compute environments. (Google AI for Developers)

**Good fit:** experimentation, smaller infrastructure, edge applications and Google-oriented ecosystems.

---

## 8.5 Phi family

Developed by Microsoft.

Typical characteristics:

- Small language models
- Designed for constrained compute
- Potential on-device or edge deployment
- Lower memory and latency requirements
- Useful for focused workloads

Microsoft positions Phi as a family of small language models intended for scenarios where compact deployment, lower latency and lower infrastructure requirements matter. (Microsoft Azure)

**Good fit:** classification, extraction, lightweight assistants, edge scenarios and high-volume focused tasks.

---

## 8.6 Model-family comparison

| Family | Weight access | Typical advantage | Main trade-off |
|---|---|---|---|
| GPT | Proprietary/API | Managed frontier capabilities | Provider dependency |
| Llama | Open weight | Self-hosting and customization | Operational burden |
| Mistral | Mixed | Efficiency and deployment choice | Model selection complexity |
| Gemma | Open weight | Lightweight Google ecosystem models | Capability depends strongly on size |
| Phi | Open, small models | Low-resource and edge workloads | Less capacity than much larger models |

### Open weight versus open source

Do not automatically use the terms interchangeably.

- **Open weight** means model parameters are available.
- **Open source** may additionally imply sufficiently open training code, data, licensing and development processes.

Always check the model's actual license before enterprise use.

---

# 9. Cost fundamentals

## 9.1 Input versus output token pricing

Hosted-model cost is commonly calculated as:

```
Input cost =
(input tokens ÷ 1,000,000) × input price per million tokens

Output cost =
(output tokens ÷ 1,000,000) × output price per million tokens

Total request cost =
input cost + output cost
```

Providers often charge different rates for:

- Input tokens
- Cached input tokens
- Output tokens
- Reasoning tokens
- Batch processing
- Fine-tuning
- Tool or modality usage

Output tokens are frequently more expensive because they require sequential decoding, although exact pricing differs by provider and model. Current model catalogs explicitly list separate input, cached-input and output rates. (OpenAI Developers)

---

### 9.2 What increases cost?

```
Long system prompts
+ large conversation history
+ too many RAG chunks
+ verbose tool results
+ long model responses
+ repeated static context
+ unnecessarily large models
```

A common production mistake is retrieving ten documents when only three are useful.

This increases:

- Token cost
- Latency
- Noise
- Hallucination risk from contradictory context

---

## 10. Latency fundamentals

LLM latency can be divided into two useful measurements.

### 10.1 Time to first token

**TTFT** is the time before the first output token arrives.

It is affected by:

- Network delay
- Provider queueing
- Model loading and routing
- Input length
- Prompt processing
- Model size
- Available hardware
- Cold starts

## 10.2 Time per output token

After the first token, the model continues decoding.

Longer answers require more sequential generation steps:

```
100 output tokens < 1,000 output tokens
```

in both latency and usually cost.

## 10.3 Prefill and decode

Inference has two conceptual phases:

### Prefill

The model processes all input tokens.

```
Longer prompt → longer prefill
```

### Decode

The model generates output one token at a time.

```
More output tokens → more decode iterations
```

Therefore:

- Input length strongly affects TTFT and memory.
- Output length strongly affects total response time.
- Streaming improves perceived latency but does not necessarily reduce total compute.

---

# 11. Product trade-offs

## 11.1 Large model versus small model

| Large model | Small model |
|---|---|
| Better capability on complex tasks | Lower latency |
| Better generalization | Lower cost |
| Usually stronger reasoning | Easier self-hosting |
| Higher GPU requirements | Better for high-volume simple tasks |
| May be unnecessary for extraction | May struggle with complex reasoning |

A practical architecture may route requests:

```
Simple classification → small model
Routine RAG question → medium model
Complex reasoning → large model
```

This is called **model routing**.

---

## 11.2 Long prompt versus retrieval

```
Send everything:
simple implementation but expensive and noisy

Retrieve selectively:
more system complexity but generally cheaper and focused
```

## 11.3 Hosted versus self-hosted

| Hosted API | Self-hosted model |
|---|---|
| Fast to integrate | Greater control |
| No GPU operations | Data can remain in your environment |
| Usage-based billing | Infrastructure-based cost |
| Provider handles scaling | Team handles scaling |
| Limited weight control | Fine-tuning and quantization flexibility |

## 11.4 Quality versus determinism

- Creative assistant: allow broader sampling.
- Financial extraction: use constrained output and low randomness.
- RAG answering: require citations and evidence.
- Agent actions: use structured outputs, validation and authorization rather than relying only on low temperature.

---

# 12. Production optimization checklist

For lower cost and latency:

1. Use the smallest model that meets the quality target.
2. Limit retrieved documents to relevant evidence.
3. Remove duplicated prompt content.
4. Cap output length.
5. Stream responses for better perceived latency.
6. Cache stable prompts, embeddings and repeated results.
7. Summarize long conversation history.
8. Batch offline workloads where supported.
9. Use quantization or optimized serving for self-hosted models.
10. Track tokens, TTFT, total latency, error rate and answer quality together.

Optimizing only cost can lower quality. Optimizing only model quality can make the product financially impractical.

---

# 13. Interview Q&A

## Q1. Why do LLMs use tokens instead of complete words?

**Answer:** Complete-word vocabularies become extremely large and cannot easily represent rare or unseen words. Subword tokens balance vocabulary size and sequence length by keeping frequent patterns together and splitting rare words into reusable pieces.

---

## Q2. What is the difference between BPE and SentencePiece?

**Answer:** BPE is an algorithm that repeatedly merges frequent adjacent symbols. SentencePiece is a tokenizer framework that learns directly from raw text and can use algorithms such as BPE or Unigram tokenization.

---

## Q3. Explain the main LLM training stages.

**Answer:** Pre-training teaches general language patterns through next-token prediction. Supervised or instruction tuning teaches the model to follow tasks and respond in useful formats. RLHF or DPO then uses preference data to make desirable responses more likely.

---

## Q4. How does DPO differ from traditional RLHF?

**Answer:** Traditional RLHF usually trains a reward model and then optimizes the LLM using reinforcement learning. DPO directly trains on preferred-versus-rejected response pairs without requiring an explicit reward model or separate RL optimization loop.

---

## Q5. What does temperature do?

**Answer:** Temperature changes the sharpness of the next-token probability distribution. Lower values favour high-probability tokens and produce more focused outputs. Higher values make lower-probability tokens more selectable, increasing diversity but also uncertainty.

---

## Q6. What is the difference between top-k and top-p?

**Answer:** Top-k allows sampling from a fixed number of highest-probability tokens. Top-p allows sampling from a dynamic set whose combined probability reaches a chosen threshold. Top-p adapts the candidate count to the model's confidence.

---

## Q7. Why does a longer context window not automatically produce a better answer?

**Answer:** A longer context may include irrelevant, conflicting or poorly positioned information. It also increases cost, prompt-processing latency and memory usage. Relevant context selection remains important even with long-context models.

---

## Q8. Does a large context window eliminate the need for RAG?

**Answer:** No. A context window determines how much information the model can receive, while RAG determines which information should be supplied. RAG is still valuable for relevance, freshness, access control, citations and large knowledge bases.

## Q9. Why are output tokens often more latency-sensitive than input tokens?

**Answer:** Input tokens are processed during the prefill phase, while output tokens are normally generated sequentially during decoding. Every additional output token requires another generation step, so long responses directly increase end-to-end latency.

---

## Q10. How would you select a model for a production application?

**Answer:** I would evaluate task quality, latency, token cost, context requirements, structured-output reliability, privacy, deployment model, licensing and operational burden. I would choose the smallest model that meets measured quality requirements and route unusually complex requests to a stronger model.

---

# Final interview summary

> “An LLM first converts text into subword tokens. During pre-training it learns next-token prediction from large datasets. Instruction tuning teaches it to follow tasks, and RLHF or DPO aligns responses with preferences. At inference time, temperature, top-k and top-p control token selection, while repetition penalties and output limits control generation behaviour. Context length, model size and token counts directly affect cost and latency. Production model selection is therefore a quality–cost–latency–control trade-off rather than simply choosing the largest available model.”